🧪 Lecture 5 — Debugging & Exceptions (30 MCQs)

🛡️ Part 1: Debugging Principles & Bug Prevention

1.

What is the primary goal of debugging in software construction?
A. To eliminate the need for testing
B. To improve performance only
C. To locate, understand, and fix faults in software
D. To rewrite the entire program

Answer: C

2.

Which strategy is considered the **first defense** against bugs?
A. Manual debugging
B. Logging runtime errors
C. Making bugs impossible through design
D. Catching exceptions

Answer: C

3.

Which of the following is an example of **static checking**?
A. Java throwing ArrayIndexOutOfBoundsException at runtime
B. Compiler detecting type mismatch errors
C. Handling exceptions using try-catch
D. Logging program output

Answer: B

4.

Dynamic checking differs from static checking because it:
A. Occurs only during compilation
B. Eliminates all bugs
C. Detects errors during program execution
D. Requires no runtime cost

Answer: C

5.

Why does Java prevent array overflow bugs using dynamic checking?
A. To improve performance
B. To make array access faster
C. To catch invalid indices at runtime
D. To avoid using assertions

Answer: C

6.

An immutable type is best described as a type whose:
A. References cannot change
B. Values cannot change after creation
C. Variables cannot be reassigned
D. Methods cannot be overridden

Answer: B

7.

Why is `String` considered immutable in Java?
A. It is stored in stack memory
B. Its value cannot be modified after creation
C. It uses dynamic checking
D. It is declared using `final`

Answer: B

8.

What is the main benefit of immutability in debugging?
A. Faster execution
B. Reduced memory usage
C. Preventing unexpected side effects
D. Easier syntax

Answer: C

9.

What does declaring a variable as `final` guarantee?
A. The object cannot change
B. The reference cannot be reassigned
C. The variable is immutable
D. The value is constant at runtime

Answer: B

10.

Why is using `final` variables considered good documentation?
A. It improves runtime performance
B. It reduces memory allocation
C. It clearly states programmer intent and is compiler-checked
D. It enables garbage collection

Answer: C

🧠 Part 2: Assertions & Bug Localization

11.

What is the main purpose of an assertion?
A. Handling user input errors
B. Replacing exceptions
C. Checking programmer assumptions during execution
D. Improving user experience

Answer: C

12.

Which of the following best describes an assertion?
A. A comment explaining code
B. Executable code that enforces assumptions
C. A compile-time directive
D. A logging mechanism

Answer: B

13.

What happens when an assertion condition evaluates to false?
A. The program continues normally
B. A checked exception is thrown
C. An AssertionError is thrown
D. The assertion is ignored

Answer: C

14.

Why are Java assertions **disabled by default**?
A. They are unsafe
B. They reduce code readability
C. They may affect runtime performance
D. They are deprecated

Answer: C

15.

Which of the following is an appropriate use of assertions?
A. Checking user input validity
B. Testing network availability
C. Verifying method preconditions
D. Handling file system errors

Answer: C

16.

Why should assertions NOT be used for external failures?
A. They cannot catch errors
B. External failures are not programming bugs
C. Assertions slow down I/O operations
D. Assertions work only with primitives

Answer: B

17.

Which statement about assertions is correct?
A. They must always be enabled in production
B. Program correctness should depend on assertions
C. Assertions should not have side effects
D. Assertions replace exception handling

Answer: C

18.

What is the main benefit of modularity in debugging?
A. Reducing memory usage
B. Making bugs easier to localize
C. Eliminating the need for testing
D. Increasing execution speed

Answer: B

19.

Encapsulation helps debugging because it:
A. Hides code from the compiler
B. Prevents bugs in one module from corrupting others
C. Eliminates runtime errors
D. Removes dependencies

Answer: B

20.

Why is minimizing variable scope important for debugging?
A. It improves performance
B. It simplifies garbage collection
C. It reduces places where a bug can affect the program
D. It avoids checked exceptions

Answer: C

⚠️ Part 3: Debugging Strategy & Exceptions

21.

What is the first step in systematic debugging?
A. Fixing the suspected code
B. Writing assertions
C. Reproducing the bug reliably
D. Refactoring the design

Answer: C

22.

Using the scientific method in debugging involves all EXCEPT:
A. Studying data
B. Forming hypotheses
C. Random code changes
D. Experimentation

Answer: C

23.

Why is regression testing important after fixing a bug?
A. To improve performance
B. To ensure no new bugs were introduced
C. To reduce test cases
D. To validate user input

Answer: B

24.

An exception in Java is best defined as:
A. A compile-time error
B. A logic error only
C. An event that disrupts normal program flow
D. A syntax violation

Answer: C

25.

What happens when an exception is thrown in Java?
A. The program immediately terminates
B. The JVM searches the call stack for a handler
C. The exception is ignored
D. The compiler fixes it

Answer: B

26.

Which exception typically indicates a programming bug?
A. IOException
B. FileNotFoundException
C. NullPointerException
D. SQLException

Answer: C

27.

Checked exceptions are primarily used to:
A. Signal programming bugs
B. Signal special results that callers should handle
C. Improve performance
D. Replace assertions

Answer: B

28.

Which rule applies to checked exceptions in Java?
A. They are ignored by the compiler
B. They must be caught or declared
C. They terminate the program immediately
D. They cannot be user-defined

Answer: B

29.

Unchecked exceptions differ from checked exceptions because they:
A. Are checked at compile time
B. Must be declared using throws
C. Typically indicate bugs and are not enforced by the compiler
D. Cannot be caught

Answer: C

30.

Which guideline is correct regarding exception design?
A. Always prefer unchecked exceptions
B. Use checked exceptions when callers cannot easily prevent failure
C. Never create custom exceptions
D. Replace return values with exceptions in all cases

Answer: B